Granite Interactive PoC — Complete Panel Explanation Guide

Source file: `markdowns/05-deliverables/poc-canvas/Granite_Interactive_PoC.html` **Everything in this guide is taken from that file.** Nothing is invented. Where the file does not say something, this guide says so rather than filling the gap.

How to use this guide

Three labels are used on every single element. Learn them before anything else, because the panel's sharpest question will be *"is that real?"*

Label	Meaning	How to say it out loud
REAL	A number or fact that comes from a committed spike output file	"That number is from a file, I can open it"
SIMULATED	A scripted animation showing how the system behaves. Nothing executes	"That's a scripted walkthrough of the mechanism"
CONCEPTUAL	A planned capability, described but not built	"That's committed scope, not built yet"

The golden rule for this demo: the PoC already labels its own simulated parts on screen. Never let the panel discover that label before you say it.

Part 1 — What this PoC actually is

1.1 The honest technical description

It is **one self-contained HTML file** that opens in a browser.

Property	Reality
Backend	**None.** No server, no database, no API
Model calls	**None.** No LLM is called anywhere
Data source	Values are **written into the JavaScript** in the file
Internet needed	Only for Google Fonts. It works offline, just with different fonts
What runs	Tab switching, an SVG animation, a constraint-check demo, chart drawing, a clickable table

Say this if asked, without hesitation: *"It's a front-end demonstration. The numbers in it are transcribed from our real spike output files, but the page itself doesn't run the pipeline — it shows what the pipeline does."*

That is not a weakness. A proposal-stage PoC is meant to show the mechanism and the evidence. Pretending it is a live system is what would damage you.

1.2 Why a PoC like this exists

The panel has to decide three things:

1. **Is the problem real?** → the Overview and Execution tabs
2. **Does the proposed mechanism make sense?** → the Silent Failure tab
3. **Is there evidence, and is the team honest about it?** → the Evidence tab

The Comparison and Scope tabs answer two follow-ups: *has someone already done this?* and *can three students finish it in a year?*

1.3 The six tabs, and the argument they make in order

```
Overview     "Here is the problem, and the headline numbers."
   |
Execution    "Here is exactly how a silent failure happens and gets fixed."
   |
Silent       "Here is the one mechanism that makes it possible."
Failure
   |
Evidence     "Here is proof we ran things, including a failure."
   |
Comparison   "Here is why nobody has already done it."
   |
Scope        "Here is why three of us can finish it in a year."
```

Each tab answers the objection the previous one raises. Present them in order.

Part 2 — Global elements (visible on every tab)

2.1 The header

Element	What it says	What it means	Label
Gold "G" logo	—	Branding only	—
Title	**Granite** / "Self-healing multi-agent diagnosis · F26-060"	Project name and official code	REAL
Chip 1	**Stream A**	Handbook classification: the idea came from published research, not an observed problem	REAL
Chip 2	**8 spikes on disk**	Eight proof-of-concept experiments with committed output files	REAL
Chip 3	**$0 committed**	Every spike ran on a local model. No paid API spend so far	REAL

Verbal line: *"Three facts before we start: research-derived project, eight experiments already run with files on disk, and it has cost nothing so far because everything ran locally."*

Watch out: "$0 committed" can be misread as *"the whole project is free."* It is not. It means **no money has been spent yet**. The full reproduction is expected to need a stronger paid model. Say "so far" every time.

2.2 The tab bar

Six buttons. Clicking one shows that panel, hides the others, and scrolls to the top. The Evidence tab also triggers its bar chart animation the first time it is opened.

Practical demo note: the bars on the Evidence tab **animate only once per page load**. If you want that animation live in front of the panel, do not open the Evidence tab while setting up. If you already did, refresh the page before presenting.

Part 3 — Tab 1: Overview

3.1 The hero statement

> **"Agents fail. Silently."** *"A wrong answer that looks completely right. No crash. No error. Nothing to investigate. Granite finds which agent, catches what has no signal, and proves a fix works before you trust it."*

What it is: the entire project in four sentences.

Why it exists: the panel must understand the failure mode in the first ten seconds, or nothing else lands.

How to explain it verbally:

> *"Normally when software breaks, it crashes and you get an error to investigate. When a team of AI agents breaks, you often get a confident, well-written, complete answer that happens to be wrong. There's no alert, because nothing technically failed. That's a silent failure, and it's what this project is about."*

3.2 "Run the pipeline" button and the mini pipeline

What it is: a button that lights up six rows one at a time, roughly 0.43 seconds apart.

What happens when pressed:

Row	Text	Side note	Visual
1	Agents run	4 agents	normal
2	**Wrong answer ships**	no signal	**turns red**
3	Constraint check	0% API	normal
4	Culprit located	agent + step	normal
5	Repair replayed	N=5	normal
6	**Recovered**	verified	**turns green**

Why it exists: it compresses the whole system into six beats before any detail arrives. The red row is the problem; the green row is the outcome.

How it connects: these six rows are the same story the Execution tab tells in twelve detailed steps. This is the trailer; that is the film.

Label: SIMULATED. It is a timed CSS animation. Nothing is computed.

Meaning of the side notes — know these, they are load-bearing:

- **"4 agents"** — matches the four agents in the Execution graph
- **"no signal"** — the definition of a silent failure
- **"0% API"** — the constraint check makes **zero** model calls. It is deterministic code
- **"N=5"** — a repair is replayed five times, not once, because model steps vary between runs
- **"verified"** — the fix is only trusted after those replays

Verbal line: *"Six steps. Agents run, a wrong answer ships with no signal, a deterministic check catches it, we locate the culprit, we replay the repair five times, and only then do we call it recovered."*

3.3 The four headline numbers

Number	Label	Caption in the PoC	What it really means	Label
71.4%	Agent accuracy	mean of 5 real runs · 62.5–85.7%	How often we identified the right **agent**, averaged over five runs	**REAL**
13.2%	Step accuracy	simplified spike · full repro pending	How often we identified the right **step**. Deliberately shown even though it is low	**REAL**
4/6	Repair success	both failures, one shared cause	Six repair attempts, four worked when replayed	**REAL**
8/8	Spikes with data	incl. 1 honest negative	All eight experiments produced committed output files	**REAL**

Why 13.2% is on the slide at all — this is the most important thing on the tab.

The best published method in the world reaches **59.5%**. We reached **13.2%**. Putting that on the front page is a deliberate credibility move.

Verbal line, and rehearse this one:

> *"You'll notice our step accuracy is 13.2% against a published 59.5%. We put it on the front page on purpose. It's a simplified two-stage version of a three-stage method, on a small local model, on eight cases. Closing that gap is the main job of the first semester. Hiding it would make every other number here worthless."*

Why "4/6" says "one shared cause": both failures made the same mistake — returning a *derived* value instead of the single atomic field asked for. One bug, not two.

Why "8/8" says "incl. 1 honest negative": one of the eight experiments **failed**. It is counted and shown anyway.

3.4 The three positioning cards

Card	Heading	Claim	What it is defending against
What	Diagnosis layer, not another tracer	Sits on top of LangSmith / Langfuse / Arize. They capture. Granite explains, repairs, verifies	*"Isn't this just another monitoring tool?"*
Why	14.2% is today's baseline	A human reading the trace and guessing. The field's own measured number	*"Can't a developer just read the logs?"*
Novel	The combination	Attribution + silent-failure detection + verified repair + unseen frameworks. No paper, product or prior FYP has all four	*"What's actually new here?"*

Label: REAL (all three are positions taken from the project's research documents).

The "What" card matters more than it looks. It stops the panel filing you under "competitor to LangSmith." You are a **layer above** those tools: they record what happened, you explain it.

Verbal line: *"Existing tools capture the trace. They're good at it. None of them tells you which step caused the failure, or whether your fix works. That's the layer we're adding."*

Part 4 — Tab 2: Execution (the centrepiece)

This is where you will spend most of the demo. Learn it properly.

4.1 The honesty banner — read it out, do not skip it

The tab carries **two** labels, both written into the page:

- Above the title: **"End-to-end · simulated from Spike 3b + 4 + 5 + 8 shapes"**
- On the graph itself, a yellow badge: **"Simulated run"**

What this means precisely: the *numbers* and the *behaviours* come from four real spikes. The *story* — a refund workflow with a stale promo table — was assembled for the demo. It is not a captured run.

Say it first, in your own words:

> *"This is a scripted walkthrough, and it's labelled as one. The mechanism at each step is real and each one has a spike behind it. The refund story is a composite we built so the whole pipeline fits on one screen."*

Why this is the right call: the four spikes each tested **one** mechanism in isolation. None of them ran end-to-end, because the pipeline is not wired together yet — that is first-semester work. This tab shows what the assembled system will do.

4.2 The two bands

The graph is split into two horizontal zones. This split *is* the architecture.

Band	Label on screen	Contains	Meaning
Top	TARGET SYSTEM — CREWAI WORKFLOW (OBSERVED)	5 nodes + 1 tool	Somebody else's AI system. We only watch it
Bottom	GRANITE — DIAGNOSIS & REPAIR	6 nodes	Our system

Why this matters more than anything else on the tab: it shows Granite does **not** modify the customer's agents. It observes, diagnoses, and hands back a verified fix. That answers "is this invasive?" before it is asked.

Verbal line: *"Top half is the customer's system — we don't touch it. Bottom half is ours. The only thing crossing between them is the trace coming down, and a verified fix going back up."*

4.3 Every node in the graph

Top band — the observed system

Node	Subtitle	Inspector details	Role
Intake Agent	parses request	Agent · Target system · CrewAI · Reads: user request · Writes: order ctx	Understands what the customer asked for
Policy Checker	validates rules	Agent · Target system · CrewAI · Calls: promo table · Writes: discount	Looks up which discount applies
Refund Calculator	computes total	Agent · Target system · CrewAI · Reads: discount · Writes: refund_total	Does the arithmetic

Node	Subtitle	Inspector details	Role
Response Writer	final answer	Agent · Target system · CrewAI · Reads: refund_total · Writes: user reply	Writes the sentence the customer reads
Output	to user	Sink · Target system · Status: **ships either way**	What the customer receives

"Ships either way" is the whole problem in two words. The answer goes out whether it is right or wrong, because nothing detects the difference.

The tool

Node	Subtitle	Inspector details
Promo Table	v2 · stale	Tool · Version: v2 (cached) · Issue: superseded by v3 · **Effect: wrong discount, no error**

This is the root of the failure. A cached lookup table that is one version out of date. It does not error — it returns a real, well-formed, *outdated* number.

Why this scenario was chosen: it is boring and completely believable. No exotic AI failure. Just stale cached data, which every engineer in the room has been burned by.

Bottom band — Granite

Node	Subtitle	Inspector details	Plain meaning
Trace Store	captured run	Data store · Granite · Holds: normalized trace · Schema: RUN / TRACE_NODE / EDGE	Where the recording of the run is kept
Graph Builder	structural parse	Component · Granite (Noor) · **LLM calls: 0** · Output: execution graph	Turns the recording into a map of what depended on what
Constraint Check	SED · 0% API	Component · Granite (Noor) · **LLM calls: 0** · Rule: refund == total − discount · Verdict: FLAG	Checks the answer against the task's own arithmetic
Causal Tracer	backward BFS	Component · Granite (Noor) · **LLM calls: 0** · Output: culprit + confidence	Walks the map backwards to find the origin
Repair Agent	candidate fix	Component · Granite (Saif) · **LLM calls: 1 per candidate** · Spike: 8 → 4/6 success	Proposes a corrected value
Verified Replay	N=5, pinned	Component · Granite (Saif) · Protocol: frozen upstream · Reports: recovery rate	Re-runs the downstream steps to check the fix works

The single most important pattern here — memorise it:

> **Four of the six Granite components make ZERO model calls.** Only the Repair Agent calls a model, once per candidate fix.

Why this matters so much: it means the diagnosis does not depend on an AI being clever. It is deterministic code. If a panelist says *"you're just asking ChatGPT to guess,"* this is the answer.

Verbal line: *"Look at the LLM-calls row on each of these. Graph building, the constraint check, the causal tracer — all zero. The only model call in the whole diagnosis is generating a candidate fix, and even that gets replayed five times before we trust it."*

Notice also the names in brackets — "Granite (Noor)", "Granite (Saif)". Ownership is visible in the architecture itself. If the panel asks who built what, the answer is on screen.

4.4 The twelve steps, one by one

Press **Play** and these run automatically, 1.45 seconds apart. Red text = something went wrong. Green = recovery.

#	On-screen message	What is happening	Colour
0	Ready. Press play.	Idle	—
1	Request arrives. Intake parses it.	A refund request comes in	—
2	Policy Checker calls the promo table.	It looks up the discount	—
3	**Tool returns v2 — stale. No error raised.**	**The fault enters here.** Cached table gives an outdated discount. Nothing errors	**RED**
4	**Refund computed from the stale discount → $20.00.**	Arithmetic done on bad input. Result is wrong	**RED**
5	Writer formats it. Fluent, confident, wrong.	Turned into a polished sentence	—
6	**Ships to the user. Nothing looks broken.**	The customer gets a wrong number	**RED**
7	Granite captures the trace, builds the graph.	Our side starts. Recording becomes a dependency map	—
8	**Constraint check: 29.75 − 10.00 ≠ 20.00 → FLAG.**	The deterministic check catches it	**RED**
9	Backward trace → culprit: Refund Calculator, step 3.	Walks backwards to the origin	—
10	Repair Agent proposes: use promo table v3.	Suggests the current table instead of the cached one	—
11	**Replayed N=5, upstream frozen → 4/5 recovered. $19.75.**	Fix replayed five times, works four times, correct value produced	**GREEN**

The arithmetic — you must be able to do this on a whiteboard:

```
order_total  = 29.75      the true order amount
discount     = 10.00      the CURRENT discount (promo table v3)
expected     = 19.75      29.75 - 10.00

stated       = 20.00      what the system actually said
                          (it used the STALE discount, 9.75)

gap          =  0.25      small enough that a human skims past it
```

This is what makes the example good. A $0.25 error looks like a rounding difference. It is not. It is a wrong discount, and at scale it is money moving on a wrong number.

4.5 The controls

Control	What it does	When to use it
Play / Pause	Auto-advances every 1.45s. Restarts from 0 if already finished	Your default. Let it run once end-to-end
Step (⏭)	Advances exactly one step and pauses	**Use this when the panel asks a question mid-demo**
Reset (↺)	Back to step 0, clears any selected node	Before a second run-through
Scrub slider	Jump to any step, 0–11	To re-show a specific moment without replaying

Control	What it does	When to use it
0 / 11 counter	Current position	—
Zoom + / −	0.6× to 2.2×	Zoom in on the Granite band if the room is large
Drag	Click and drag to pan	Only useful when zoomed in

Presentation tip: rehearse with **Step**, not Play. Play is for the first pass; Step is for answering questions. If you press Step while playing, it pauses automatically.

4.6 The moving dots

Small circles travel along an edge each time a step fires. They are **colour-coded**:

Colour	Meaning	Which steps
Gold	Normal data flow	1, 2, 5, 7, 9, 10
Red	Corrupted data moving	3, 4, 6, 8
Green	Verified fix returning	11

Why this exists: it makes *propagation* visible. The panel can literally watch the wrong value travel from the tool, through two agents, out to the user. That is the concept "error propagation" without a word of jargon.

4.7 The node inspector (right-hand panel)

What it is: click any node and the panel on the right fills with that node's details and its connections.

What happens visually: the selected node gets a teal outline. Its directly connected neighbours stay bright. **Everything else fades to 30% opacity.** Edges touching it turn teal.

Why it exists: it answers "what does this component actually depend on?" on demand, without another diagram.

How to use it in the demo — this is your best interactive moment:

> Let the panel pick. *"Click any box and it'll tell you what it is, who owns it, how many model calls it makes, and what it's connected to."*

If nobody picks, click **Constraint Check** yourself. It shows `LLM calls: 0` and `Rule: refund == total − discount` — your headline claim, in the architecture, on demand.

Click the same node again to deselect.

4.8 The legend

Tag	Meaning
agent	Runs in the observed system
tool / store	External dependency
fault	Wrong, but no error raised
verified	Replay confirmed recovery

The "fault" definition is the important one: **wrong, no error raised**. That is a silent failure in four words.

4.9 Two corrections made to this tab — and why they matter

Both of these were real inconsistencies. **Both are now fixed in the file.** They are recorded here because the *reasoning* is worth knowing — a panelist may ask about attribution logic directly, and this is the answer.

Correction 1 — the named origin now matches the proposed fix

What it used to say: step 9 — *"Backward trace → culprit: **Refund Calculator**, step 3."* But step 10 proposed fixing the **promo table**.

Why that was wrong. Work the arithmetic:

```
stated  = 20.00   requires  discount = 9.75   (29.75 − 9.75 = 20.00)
true    = 19.75   requires  discount = 10.00  (29.75 − 10.00 = 19.75)
```

The Refund Calculator subtracted **correctly**. It was handed a stale discount and did exactly the right arithmetic on a wrong input. The origin is the **Promo Table**.

Why it mattered so much: blaming the component where the wrong value *appears* instead of where it *originated* is precisely the mistake Granite exists to prevent. A panelist could have said *"your own demo attributes to the wrong agent."*

What it says now:

> *"Backward trace → origin: Promo Table (stale v2), surfaced at Refund Calculator."*

The Promo Table node now also lights up at this step, so the graph and the text agree.

The concept underneath — worth being able to explain: attribution distinguishes the **origin** (where the fault entered) from where it **surfaced** (where it became visible). Fixing the place it surfaced would not help. That distinction is the entire value of backward causal tracing.

> *"The wrong number shows up at the Refund Calculator, but that agent did its job correctly — it subtracted a stale discount. Tracing backwards gets us to the actual origin, the cached promo table. If we'd stopped at the first place the number looked wrong, we'd have fixed the wrong component."*

Correction 2 — the replay step no longer quotes an unmeasured ratio

What it used to say: step 11 — *"Replayed N=5, upstream frozen → **4/5 recovered**."*

Why that was wrong: Spike 5 was a **single** deterministic replay (97.2 → 108.0). Spike 8 measured **4/6** on repair generation. **No spike produced a 4-out-of-5 result.**

What it says now:

> *"Replayed N=5, upstream frozen → recovery rate measured. $19.75."*

This keeps the protocol point — replay N times, report a rate — without quoting a number no file supports.

If asked why N replays instead of one:

> *"Because model steps aren't deterministic. The same input can give different output, so a single replay could be luck. We report a recovery rate over N runs with its variance. Our measured numbers so far are Spike 5's deterministic replay and Spike 8's four-out-of-six repair rate."*

Part 5 — Tab 3: Silent Failure (the headline claim)

Labelled on screen as **"The headline claim."** If the panel remembers one tab, make it this one.

5.1 The argument in one line

> *"Every normal check passes. The answer is still wrong. A deterministic constraint check — zero model calls — is what separates the two."*

5.2 Left card — "What every existing check sees"

Five checks, each animating to a green ✓ about 0.15 seconds apart:

#	Check	Why it passes on a silent failure
1	No exception thrown	The code ran fine. Wrong arithmetic is still valid arithmetic
2	Schema valid	The output has the right shape. `$20.00` is a perfectly good number
3	All fields present	Nothing is missing
4	Fluent, confident tone	Language models are *always* fluent. Fluency carries no information about correctness
5	Tool call succeeded	The promo table **did** respond. It just responded with stale data

Ending line: **"→ Ships to the user. Nothing to investigate."**

Why this card exists: it proves the problem is not laziness or bad monitoring. **Every reasonable check genuinely passes.** The failure is invisible to all of them by construction.

Verbal line — deliver it slowly:

> *"Five checks any sensible engineer would run. All five pass. And the answer is still wrong. That's why this isn't solved by better monitoring — there's nothing for monitoring to see."*

Check 4 is the one to dwell on. *"An LLM is fluent whether it's right or wrong. Confidence in the writing tells you nothing about correctness — that's exactly why these failures survive human review."*

5.3 Right card — "What the constraint check sees"

A code block showing the actual rule:

```
// task spec yields a checkable rule
assert refund_total == order_total - discount
// evaluate
order_total = 29.75
discount    = 10.00
expected    = 19.75
stated      = —          ← filled in when you press a button
```

What it is: the real logic of the Semantic Error Diagnostic, written out.

Why it exists: it shows the mechanism is **simple, not clever**. Recompute the answer from the task's own numbers and compare. No AI judgment involved.

The first line is the most important one. *"The task spec yields a checkable rule."* This is the boundary of the entire claim. Where a rule can be derived, the check works. Where it cannot, the checker **abstains**.

Verbal line:

> *"This is the whole mechanism. The task itself tells us what the answer should be — order total minus discount. We compute it independently and compare. There's no model call and no judgment. It's arithmetic."*

5.4 The two buttons — the interactive proof

Button	What happens	Verdict shown
"Feed a wrong value"	Five checks pass, then `stated` becomes **20.00**	**FLAG — constraint violated.** *"Off by $0.25. Plausible enough to pass a human skim. Caught deterministically, zero model calls."*
"Feed a correct value"	Five checks pass, then `stated` becomes **19.75**	**PASS — constraint satisfied.** *"Correct value passes. This is the false-positive check — the detector stays quiet when nothing is wrong."*

Below them: *"Both paths are real detector behaviour — flag on wrong, pass on correct."*

Label: SIMULATED — the page does not run the real checker; it shows the two outcomes the real checker produces.

Why the "correct value" button is as important as the wrong one — do not skip it:

A detector that flags everything catches every failure and is useless. The **PASS** path proves the checker stays quiet when nothing is wrong. That is the false-positive check.

Demo instruction: press BOTH buttons, wrong first, then correct.

> *"Wrong value — flagged. Now the same detector on a correct value — it passes. That second one matters as much as the first. A detector that flags everything would score perfectly on catching failures and be completely useless. This is the check that stops us fooling ourselves."*

5.5 The two evidence cards — the most honest thing in the PoC

Card	Tag	What it says	Cited source
Mechanism test	green — **passed**	Fed known-wrong values including a plausible near-miss (**$20.00** vs true **$19.75**): flagged every one. Correct values passed. **Zero false positives**	risk-register R10
Natural-occurrence run	amber — **inconclusive, by design**	n=3 live tasks: the model got all 3 **right**, so there was no natural silent failure to catch. `n_true_silent_failure_cases: 0`. Reported as-is, not hidden	`poc_spike4_silent_failure_output.json`

This split is the strongest credibility move in the whole PoC. Understand it exactly.

Two different tests, and they must never be blurred:

1. **Mechanism test** — we hand-fed the checker values we knew were wrong. It caught every one. This proves **the mechanism works**.
2. **Live run** — we gave a real model three tasks, hoping it would make a natural mistake. **It got all three right.** So there was nothing to catch, and the live run proves nothing about how often silent failures occur.

Verbal line — rehearse word for word:

> *"Two different runs, and I want to be precise about them. The mechanism test: we fed the checker known-wrong values including a near-miss, twenty dollars against a true nineteen seventy-five. It caught every one, with no false positives on the correct values. The live run: three real tasks, and the model got all three right — so there was no natural failure to catch. That run is inconclusive and we've labelled it inconclusive. Our real-world catch rate is unmeasured, not high."*

Note for accuracy: the card cites `risk-register R10`. That mechanism test now has its own committed output file, `poc_spike4b_mechanism_output.json`. Citing the file is stronger than citing a register entry — worth updating.

5.6 Scope of the claim

> *"Works where the task spec yields a machine-checkable rule — numeric totals, required fields, policy sequences. Where none exists the checker **abstains** rather than guessing, and the abstention rate is reported as its own metric. Not a general 'understands meaning' claim."*

What it is: a pre-emptive limit on the claim.

Why it exists: to stop the panel asking *"so you can detect any wrong answer?"* The answer is **no**, and saying so first is what makes the rest believable.

Three terms, defined simply:

- **Machine-checkable rule** — something a computer can verify without judgment. "Total minus discount" is checkable. "Is this a good essay" is not.
- **Abstains** — when there is no rule, the checker says *"I can't check this"* instead of guessing.
- **Abstention rate** — how often it has to say that. Reported as a metric, so the coverage of the claim is measured rather than assumed.

Verbal line:

> *"We're not claiming to understand meaning. We're claiming that where the task gives us a checkable rule, we catch what nobody else catches — and where it doesn't, we say so and count how often. That's a boundary we measure, not one we hope nobody asks about."*

Part 6 — Tab 4: Evidence

Labelled: **"Every number recomputable from `dev/spikes/poc/`."**

6.1 The five-run bar chart

Five bars, one per run. Each row shows agent % and step %:

Run	Agent	Step
Run 1	62.5%	0.0%
Run 2	75.0%	25.0%
Run 3	71.4%	14.3%
Run 4	62.5%	12.5%
Run 5	85.7%	14.3%

Label: REAL. These match `poc_spike1_multirun_summary.json` exactly.

Note: the coloured **bar** shows the agent number only. The step number is text beside it.

Why five runs and not one: the same code on the same eight cases gives different answers each time, because language models are not deterministic. **Run 1 got 0% on step accuracy. Run 2 got 25%.** Reporting one run would be reporting a lucky number.

Verbal line:

> *"Five identical runs, five different results. Agent accuracy from 62 to 86 percent, step accuracy from zero to twenty-five. That's why we report a range. If we'd run it once and got Run 5, we'd be claiming 85.7% — and it would be meaningless."*

The caption: *"A step-index type bug was found and fixed, then all 5 re-run."*

Be ready for this one. During the build, a bug compared a step index stored as text against one stored as a number, so correct answers were scored as wrong. It was found, fixed, and **all five runs redone** rather than patching the numbers.

> *"We found a type bug in our own scoring — comparing a text step-index against a numeric one. We fixed it and re-ran all five rather than hand-correcting the output. That's why the numbers here are lower than an earlier draft of ours."*

6.2 The published-accuracy chart

Three bars: **Who&When 14.2%** · **AgenTracer ~40%** · **ErrorProbe 59.5%**, with *"37–43% published range, midpoint shown"* under AgenTracer.

Caption: **"40.5 points still wrong at the top of the field — and none of it sees silent failures."**

Label: REAL (each from that paper's own results table).

Why the AgenTracer footnote exists: that paper publishes a **range**, not a single number. Showing a midpoint and saying so is honest; showing 43% and calling it their score would not be.

Verbal line:

> *"This is where the field is. Fourteen percent when a human reads the trace. Forty for a trained model. Fifty-nine and a half for the best published method. That top bar still gets four steps in ten wrong — and by its own admission, none of these see silent failures at all."*

6.3 The eight-spike table

#	Spike	Status	Result
1	ErrorProbe reproduction	demonstrated	agent 62.5–85.7% · step 0–25% · 5 runs, n=8
2	CrewAI instrumentation	demonstrated	3 nodes, 2 clean edges — framework no paper used
3/3b	Fault injection	demonstrated	corrupted tool output · 1 case self-corrected, kept as a finding
4	Constraint check (SED)	demonstrated	flags known-wrong incl. near-miss ($20.00 vs $19.75) · 0 false positives · natural-occurrence rate still unmeasured (n=3)
5	Counterfactual replay	demonstrated	97.2 corrupted → 108.0 recovered · recovery_success = true
6	Cyclic graph handling	demonstrated	traversal terminates under bound; control run proves bound load-bearing
7	Preventive classifier	**honest negative**	precision = recall = F1 = 0.25 — near-random, reported not hidden
8	Repair Agent	demonstrated	4/6 success · 0 parse failures · both failures share one cause

Label: REAL — every row matches a committed output file.

Three rows deserve special attention:

Row 3/3b — "1 case self-corrected, kept as a finding." We injected a fault and the agent **noticed and fixed it by itself.** That was not what we expected. We kept it and reported it, because real agent behaviour includes self-correction and a benchmark that ignores that would be wrong.

Row 7 — the honest negative. We tested an idea from another paper (catch errors before they propagate). It scored 0.25 — barely better than guessing. **We kept it in the table and marked it red.**

> *"Spike 7 failed. Precision, recall and F1 all 0.25, which is near-random. We're showing it because a proof-of-concept that disproves an assumption is worth more than one that confirms what you already believed — and because it's the reason our committed path is the reactive one, not the preventive one."*

Row 6 — "control run proves bound load-bearing." We tested cycle handling **twice**: once with the safety limit on (it terminated correctly) and once with it off (it ran to a 10,000-step cap). The second run is the control — without it, "our traversal terminated" would be a fact about that particular graph, not about our design.

Correction 3 — Spike 4 now reads "partial" on both tabs

What it used to say: the Evidence table marked Spike 4 **"demonstrated"** (green), while the Silent Failure tab called the same spike **"inconclusive, by design"** (amber). Two tabs grading one spike differently.

What it says now: "partial" (amber), matching the Silent Failure tab, with the detail line reading *"**mechanism** flags known-wrong… natural-occurrence rate still unmeasured (n=3)."*

Why "partial" is the right word: the mechanism is demonstrated. The occurrence rate is not. "Partial" says both at once, and it is the label you can defend without a paragraph of qualification.

> *"Spike 4 is marked partial deliberately. The mechanism works — it caught every known-wrong value with no false positives. How often silent failures actually occur in the wild is unmeasured, because the live run produced none. Partial is the honest grade."*

Part 7 — Tab 5: Comparison

Labelled: **"Click a capability to see who has it."** Headline: **"The gap is the combination."**

7.1 The matrix

Six systems × four capabilities. ✓ = has it, ✗ = does not, ? = unconfirmed.

System	Step attribution	Silent-failure	Verified repair	Cross-framework
ErrorProbe (ACL 2026)	✓ 59.5%	✗ admits it cannot	✗	✗ 2 archs only
LangSmith Engine / Arize Alyx	? not published	? not confirmed	? not confirmed	✗ stack-locked
Langfuse / Braintrust / Opik	✗ human-driven	✗ RAG-only	✗	✗
Closest prior FYP (S24-011)	✗ no attribution	? single-LLM text	✗ no replay	✗ not multi-agent
Manual workflow today	✗ 14.2%	✗	✗	✗
Granite (proposed) — gold row	✓ building on 59.5%	✓ scoped + measured	✓ recovery over N	✓ CrewAI + LangGraph

Label: REAL (positions from the literature and commercial reviews). **Granite's own row is CONCEPTUAL** — it describes what we are building, not what is measured. Say so.

Read the shape, not the cells: every competitor has some ✓s. **Nobody has four.** That is the argument — the gap is the *combination*.

The "?" marks are deliberate and defensible. LangSmith Engine and Arize Alyx ship automated diagnosis but publish no accuracy number on any benchmark. We mark that unknown rather than claiming they cannot do it. **Never upgrade a "?" to a "✗" in conversation** — it would be a claim you cannot support.

7.2 The interactive column filter

Click any column header. It highlights that column and swaps the note underneath:

Column	Note that appears
Step attribution	"Only Granite publishes a benchmarked step-accuracy number on unseen architectures."
Silent-failure	"Silent-failure detection: admitted blind spot in the strongest paper; RAG-only or unconfirmed elsewhere."
Verified repair	"Nobody else measures whether a proposed fix actually recovers the task."
Cross-framework	"Every automated competitor is locked to its own tracing stack."

Click again to clear.

Why it exists: it lets you answer *"but what about X?"* by isolating exactly the capability being questioned, instead of gesturing at a wall of ticks.

Demo instruction: click **"Verified repair"**. It is the emptiest column — a full row of ✗ and ? with a single ✓ at the bottom.

> *"Click any capability and it isolates it. This one — verified repair — is the clearest. Nobody measures whether a proposed fix actually works before applying it."*

7.3 The three "closest" cards

Card	Named	What it says
Closest paper	ErrorProbe · ACL 2026	59.5% — best published. Its own limitations section admits it cannot see silent failures, and it has never run on an architecture outside its benchmark's own two
Closest products	LangSmith Engine · Arize Alyx	Both ship automated diagnosis (2026). Neither publishes an accuracy number on any benchmark. Both locked to their own tracing stack
Closest prior FYP	S24-011-D-HDLM · Spring 2024	Hallucination detection in single-LLM text. Same core problem, ~1 of our 4 stages. Checked structurally against 660 titles, 12 cohorts since 2019

Why these cards matter: they show you went looking for competition and **named the closest thing in each category** rather than waiting to be caught.

The prior-FYP card is the one that scores points. Volunteer it:

> *"We checked every prior FYP — 660 titles across twelve cohorts back to 2019, structurally, not by title. The closest is a Spring 2024 hallucination-detection project. It confirms the problem is real and department-recognised. It covers roughly one of our four stages, on a single model's text — it has no notion of which agent or which step, because those don't exist unless several agents are interacting."*

Correction 5 — the ErrorProbe wording was tightened. It used to read *"outside its benchmark's own two,"* which is loose: ErrorProbe evaluated on three benchmarks and three backbone families. It now reads *"never run on a multi-agent framework absent from its own benchmark ecosystem"* — which is the claim the literature actually supports. Use that phrasing verbally too.

Part 8 — Tab 6: Scope

Labelled: **"Feasibility, ownership, and the complexity case."** Headline: **"Sized for one year."**

Sub-line: *"Ten committed mechanisms, three separable lanes, five approval gates, 1,130 hours against 1,130 of capacity."*

8.1 Complex computing characteristics — 8 cleared

Eight items, each with a green ✓:

Conflicting requirements · No obvious solution · Depth of knowledge · Unfamiliar issues · Beyond standard practice · Significant consequences · Interdependence · Ill-defined cause

Label: REAL (matches the project's own self-rating).

Correction 4 — the ninth characteristic is now stated on the page.

The heading used to read *"8 cleared"* with the ninth silently absent. It now reads **"8 of 9 cleared"**, followed by an amber note:

> *"9th — **Diverse stakeholders:** self-rated **weak**, not claimed. Operators, downstream product teams and auditors are named, but no working engineer has used the system yet, because there is nothing running to show one. Scheduled for the Mid-2 acceptance study. Declared open rather than overclaimed."*

Why this change is worth more than it costs. A silently missing ninth looks like concealment when discovered. A ninth that is named, rated weak, and given a date looks like self-assessment. **Same fact, opposite impression.**

Say it anyway, do not just let them read it:

> *"You'll notice eight, not nine. The ninth is diverse stakeholders, and we rate it weak — we haven't put this in front of a working engineer yet, because we don't have a working system to show one. That's scheduled for the Mid-2 acceptance study. We'd rather show eight we can defend than nine we can't."*

8.2 The three ownership lanes

Owner	Lane	Responsibilities	"If removed"
Shahoud	Data / Benchmark (teal)	Framework instrumentation (CrewAI, LangGraph) · Fault injection + labelling · Benchmark build & release	No cross-architecture benchmark — the core claim is untestable
Noor	Diagnosis Engine (gold)	Graph construction + trace normalization · ErrorProbe reproduction · Constraint check + causal tracing	No attribution, no silent-failure detection — the contribution disappears
Saif	Platform (blue)	Counterfactual replay + recovery · Framework-agnostic SDK, FastAPI, Postgres · Dashboard, containerized deployment	Nothing installable outside the dev machine

Why the "If removed" line exists: it is the handbook's own test that each member owns something whose absence would break the project. Answering it in red, on screen, is stronger than claiming "we all contributed equally."

These lanes match the Execution graph. The Granite nodes are tagged "(Noor)" and "(Saif)" in their inspectors. Ownership is consistent across tabs — a good thing to point out.

Demo instruction: each member says their own lane out loud. Three voices, three distinct subsystems.

8.3 The five gates

Gate	Stage	Requirement
Gate 1 — NOW (highlighted gold)	Proposal	Scope + PoC approved
Gate 2/3	Mid-1	Full reproduction · Tier-2 gates
Gate 3	Final-1	Integrated core, end to end
Gate 4	Mid-2	Benchmark ≥500 · release ready
Gate 5	Final-2	Full evaluation, deployed

Why it exists: it shows you know what you are being graded on next, not just eventually. Gate 1 is highlighted — *"this is today."*

8.4 The three tier cards

Card	Count	Meaning
Tier 1 — evidence-backed	**8** (green)	Every one has a spike behind it. Committed unconditionally
Tier 2 — gated at Mid-1	**2** (amber)	Recovery Controller (no PoC yet) · Repair Memory (one test, failed). Pre-registered fallbacks either way
Extensions — after core works	**8** (grey)	Gated behind the committed core. **Not promised**

The Tier 2 card is the most honest card in the PoC, and you should point at it yourself.

It states in the demo, unprompted, that one core feature has **no proof-of-concept** and another has **one test that failed**.

> *"Ten committed mechanisms, but they're not equal. Eight have a spike behind them. Two don't — the Recovery Controller has no PoC at all, and the Repair Memory has one test that failed. Both have written decision gates at Mid-1 and a recorded fallback if they don't pass. We wrote those rules before we had results, which is the only time writing them means anything."*

"Pre-registered" is the key word. The pass/fail rules were written **before** the results existed, so they cannot be adjusted afterwards to make a bad result look acceptable.

Part 9 — Complete end-to-end explanation

This is the whole PoC as one continuous story. If you can tell this without the screen, you know it.

9.1 The problem

A customer asks an AI system for a refund amount. The work is split between four agents: one understands the request, one looks up the discount, one does the arithmetic, one writes the reply.

The discount lookup uses a **cached promo table that is one version out of date**. It does not error. It returns a real, well-formed, outdated number.

The Refund Calculator does its arithmetic **correctly** — on the wrong discount. The result is **$20.00** when the true answer is **$19.75**.

The Response Writer turns it into a fluent, confident sentence. It ships.

Nothing failed. No exception, no schema violation, no missing field, no failed tool call. Every check an engineer would write passes. The customer gets a wrong number and nobody knows.

That is a silent failure, and it is the reason this project exists.

9.2 What Granite does about it

Step one — capture. Granite records the run into a Trace Store. It does not change the customer's agents; it watches them.

Step two — build the map. The Graph Builder turns the recording into an execution graph: nodes are actions, edges are dependencies. **Zero model calls** — this is structural parsing.

Step three — check the answer. The Constraint Check derives a rule from the task itself: `refund_total == order_total - discount`. It computes `29.75 - 10.00 = 19.75`, compares against the stated `20.00`, and **flags it**. **Zero model calls.** This is the mechanism nothing else in the field has, and it runs on **every** trace — not only ones that look broken.

Step four — find the origin. The Causal Tracer walks the graph backwards from the flagged answer. **Zero model calls** — graph traversal. It arrives at the stale promo table, which is where the wrong number entered.

Step five — propose a fix. The Repair Agent suggests using promo table v3 instead of the cached v2. This is the **only** model call in the pipeline, and it produces a candidate, not a decision.

Step six — prove the fix works. Verified Replay re-runs the downstream steps with the corrected discount, with the upstream context frozen so nothing else changes. It runs **N times**, not once, because model steps vary. It reports a **recovery rate**. Only then is the fix trusted.

Result: the wrong answer is caught, explained, corrected, and the correction is verified — **and none of it depended on an AI being clever.** Five of the six stages are deterministic code.

9.3 Why this is not already solved

- **The best published method reaches 59.5%** step accuracy after fifteen months and seven papers. It gets four steps in ten wrong.
- **That same method admits, in its own limitations section, that it cannot see silent failures.** It looks for something visibly wrong first. When nothing looks wrong, the search never starts.
- **Commercial tools capture traces beautifully** but leave a human to work out which step was responsible.

- **Nobody measures whether a proposed fix actually works** before applying it.

Individually, pieces of this exist. **All four together, in one pipeline, do not.**

9.4 What we have proved so far

Eight experiments, all with committed output files.

Working: we can instrument a real framework and capture a graph (Spike 2). We can inject a fault with exact ground truth (Spike 3/3b). The constraint checker catches known-wrong values including a near-miss, with zero false positives (Spike 4). A corrupted value can be replayed back to correct (Spike 5). Cyclic graphs terminate under a bound, with a control run proving the bound matters (Spike 6). The repair agent works 4 times in 6, and both failures share one fixable cause (Spike 8).

Not working, reported anyway: the preventive classifier scored 0.25 — near-random (Spike 7). It stays in the table, marked red.

Honestly incomplete: our step accuracy is 13.2% against a published 59.5%. And the live silent-failure run was inconclusive, because the model got all three tasks right so there was nothing to catch.

9.5 Why it is a year of work

Ten committed mechanisms. Eight have a spike behind them; **two do not, and we say which**. Three separable lanes, one per member, each with a stated consequence if that member's work disappears. Five approval gates with defined criteria. 1,130 hours budgeted against 1,130 available.

Part 10 — Presentation script

Target: 7–9 minutes. Times are per section.

Before you start

- [ ] Open the file and **refresh** — the Evidence bars animate once per load
- [ ] Stay on the **Overview** tab
- [ ] Zoom the browser so the smallest text is readable from the back row
- [ ] Know who is clicking. One driver, not three

[0:00–0:45] Overview — the problem

> *"This is Granite. Before I show you anything, one sentence about the problem.*
>
> *When normal software breaks, it crashes and you get an error to investigate. When a team of AI agents breaks, you often get a confident, well-written, complete answer that happens to be wrong. Nothing crashes. There's no alert. That's a silent failure."*

[Click "Run the pipeline"] — let the six rows animate.

> *"Six steps. Agents run. A wrong answer ships with no signal — that's the red one. A deterministic check catches it. We locate the culprit. We replay the repair five times. Only then do we call it recovered."*

[0:45–1:30] Overview — the numbers

> *"Four numbers, all from files on disk. Agent accuracy 71.4% across five runs. Repair success four out of six. Eight spikes, all with real output.*
>
> *And the second one — step accuracy, 13.2%. The best published method in the world gets 59.5%. We're putting our 13.2% on the front page on purpose. It's a simplified version of a three-stage method on a small local model. Closing that gap is the main job of next semester. Hiding it would make every other number here worthless."*

[1:30–3:30] Execution — the flagship

[Click "Execution"]

> *"This is a scripted walkthrough and it's labelled as one — 'simulated run', top right. Each mechanism has a spike behind it. The refund story is a composite so the whole pipeline fits on one screen.*
>
> *Two halves. Top is the customer's system — four CrewAI agents and a tool. We don't touch it. Bottom is Granite."*

[Press Play. Narrate over it.]

> *"Request comes in… the Policy Checker looks up a discount… and here — the promo table returns version two. It's cached, it's stale, and it does not error. That's where the fault enters.*
>
> *The Refund Calculator does its arithmetic correctly on that wrong discount. Twenty dollars instead of nineteen seventy-five. The Writer makes it fluent. It ships. Nothing looks broken.*
>
> *Now Granite. We capture the trace and build the graph. The constraint check: twenty-nine seventy-five minus ten does not equal twenty. Flagged — with zero model calls. We trace backwards to the origin. The Repair Agent proposes the current promo table. We replay it with the upstream frozen, and we get nineteen seventy-five back."*

[Click "Constraint Check" node]

> *"You can click any box. This one's our headline component — zero LLM calls, and there's the rule it applies. Four of our six components make no model calls at all. The diagnosis doesn't depend on an AI being clever."*

[3:30–5:00] Silent Failure — the claim

[Click "Silent Failure"]

> *"Here's why this is hard. Five checks any sensible engineer would write — no exception, valid schema, all fields present, fluent tone, tool call succeeded."*

[Press "Feed a wrong value"] — let the five checks go green.

> *"All five pass. And the answer is still wrong. That's the point — this isn't solved by better monitoring, because there's nothing for monitoring to see.*
>
> *Our check is different. The task itself tells us what the answer should be: total minus discount. We compute it independently and compare. Twenty dollars — flagged. Off by twenty-five cents, plausible enough that a human skims past it."*

[Press "Feed a correct value"]

> *"Same detector, correct value — it passes. That matters as much as the first one. A detector that flags everything would catch every failure and be useless."*

[Point at the two evidence cards]

> *"And here's where we're careful. Two different runs. The mechanism test caught every known-wrong value with no false positives. The live run — three real tasks, and the model got all three right, so there was no natural failure to catch. That run is inconclusive and we've labelled it inconclusive. Our real-world catch rate is unmeasured, not high."*

[5:00–6:15] Evidence

[Click "Evidence"] — bars animate.

> *"Five identical runs. Five different results. Step accuracy from zero to twenty-five percent. That's why we report a range — one run would have been a lucky number.*

> *And this is where the field is: fourteen percent for a human reading the trace, forty for a trained model, fifty-nine and a half for the best published method. That top bar still gets four steps in ten wrong, and by its own admission sees no silent failures at all.*
>
> *Eight spikes below. Spike seven is red — a preventive classifier we tested that scored near-random. We kept it in, because a proof-of-concept that disproves an assumption is worth more than one that confirms what you already believed."*

[6:15–7:15] Comparison

[Click "Comparison"], then **[click the "Verified repair" column header]**

> *"Six systems, four capabilities. Everyone has some. Nobody has four. The gap is the combination, not one missing feature.*
>
> *This column is the clearest — nobody measures whether a proposed fix actually works before applying it.*
>
> *And we checked prior FYPs — 660 titles, twelve cohorts back to 2019. The closest is a Spring 2024 hallucination-detection project. It covers about one of our four stages, on a single model's text."*

[7:15–8:15] Scope

[Click "Scope"]

> *"Ten committed mechanisms, three lanes, five gates, eleven hundred hours against eleven hundred of capacity.*
>
> *Eight complex-computing characteristics — you'll notice eight, not nine. The ninth is diverse stakeholders and we rate it weak, because we haven't put this in front of a working engineer yet. That's scheduled for Mid-2.*
>
> *Three lanes, one each."*

[Each member names their own lane]

> *"And the honest part — ten mechanisms, but not equal. Eight have a spike behind them. Two don't: the Recovery Controller has no PoC at all, and the Repair Memory has one test that failed. Both have written decision gates at Mid-1 and a recorded fallback. We wrote those rules before we had results."*

[8:15–8:45] Close

> *"To summarise. AI agents fail silently — confident, well-formed, wrong. The best published method admits it can't see those failures. We add three things: a check that runs on every trace whether or not anything looks broken, a repair that's verified by replay before it's trusted, and testing on two frameworks no paper has used.*
>
> *Eight experiments already run, all with files on disk, including one that failed. Happy to open any of them."*

Part 11 — Likely panel questions

About the PoC itself

Q. Is this a working system?

> No. It's a front-end demonstration. The numbers are transcribed from our real spike output files, but this page doesn't run the pipeline — it shows what the pipeline does. The pieces exist as eight separate experiments; wiring them together end-to-end is first-semester work.

Q. So what part is real?

> Every number is real and traceable to a committed file. The animation is scripted. The page itself labels the simulated parts — top-right of the execution graph.

Q. Why not demo the actual code?

> The eight spikes are real Python and they run — but each tests one mechanism in isolation, and they take minutes each on a local model. This shows how they fit together. We can run any individual spike for you.

Q. Where does the refund story come from?

> It's a composite. The arithmetic is from Spike 4 — the twenty-dollar versus nineteen-seventy-five near-miss is our real test case. The four-agent refund workflow was assembled so the whole pipeline fits on one screen.

About the mechanism

Q. How is your check different from a unit test?

> A unit test is written by a developer in advance for a known case. Ours derives the rule from the task specification at diagnosis time, for a task nobody wrote a test for. Same spirit — very different reach.

Q. What if there's no checkable rule?

> It abstains — says "I can't check this" rather than guessing — and we report how often that happens as its own metric. That's the boundary of the claim, and we measure it instead of hoping nobody asks.

Q. Can you detect any wrong answer?

> No, and we don't claim to. Only where the task yields a machine-checkable rule — numeric totals, required fields, policy sequences. Not "is this a good essay."

Q. Isn't this just asking an LLM if the answer is right?

> The opposite. Four of our six components make zero model calls. The check is arithmetic, the graph builder is structural parsing, the tracer is graph traversal. The only model call is generating a candidate fix, and that gets replayed before we trust it.

Q. Why replay five times instead of once?

> Because model steps aren't deterministic — the same input can give different output. One replay could be luck. We report a recovery rate over N runs, with its variance.

About the evidence

Q. Your step accuracy is 13.2%. Theirs is 59.5%. Explain.

> That's a simplified two-stage reproduction of a three-stage method, on a local three-billion-parameter model, on eight cases. It's not like-for-like and we don't present it as such. Closing the gap is the top priority next semester, and we've pre-registered an acceptance band of plus or minus ten points before running it.

Q. Why do the five runs differ so much?

> Language models aren't deterministic. Same code, same eight cases, five different results. That's exactly why we report a range instead of one number.

Q. Your silent-failure spike caught nothing.

> Correct, and we've labelled that run inconclusive on the page. Two different runs: the mechanism test caught every known-wrong value with zero false positives. The live run had no natural failure to catch, because the model got all three tasks right. Our real-world catch rate is unmeasured, not high.

Q. Why is a failed spike in your evidence table?

> Because it's evidence. It's why our committed path is the reactive one and not the preventive one. A PoC that disproves an assumption is worth more than one that confirms what you already believed.

Q. What's the step-index bug you mention?

> Our scoring compared a step index stored as text against one stored as a number, so correct answers scored as wrong. We found it, fixed it, and re-ran all five runs rather than hand-correcting the output.

About novelty

Q. What's actually new?

> Not the attribution algorithm — we adopt the published one. Three things: a silent-failure check that isn't gated on something looking broken, a repair verified by measured replay, and testing on two frameworks no reviewed paper used. The gap is the combination.

Q. Isn't this just gluing existing techniques together?

> Partly, yes — and external components are expected. The gluing isn't the claim. Strip away every technique and what's left is a number: either the pipeline holds up on a framework it was never tuned for, or it doesn't. Nobody has produced that number.

Q. LangSmith and Arize already do automated diagnosis.

> They do, since 2026, and we mark them "unconfirmed" rather than "can't" — neither publishes an accuracy number on any benchmark, and both are locked to their own tracing stack. We're a layer above those tools, not a competitor.

Q. A prior FYP already did hallucination detection.

> S24-011, Spring 2024. We found it ourselves in a 660-title check. It confirms the problem is real. It covers about one of our four stages on a single model's text — it has no notion of which agent or which step, because those concepts need several agents interacting.

About feasibility

Q. Can three students finish this in a year?

> Ten mechanisms, eight already with a spike behind them. Three separable lanes, one per member. Eleven hundred hours budgeted against eleven hundred available. And a pre-registered descope ladder if we slip — we know what gets dropped first, in what order.

Q. Which part worries you most?

> Two. Reproducing 59.5%, and the Recovery Controller, which has no PoC at all. The Repair Memory has one test and it failed. Both Tier-2 items have written gates at Mid-1 and a recorded fallback.

Q. Why only eight CCP characteristics?

> The ninth is diverse stakeholders and we rate it weak. We haven't put this in front of a working engineer, because we don't have a working system to show one. That's scheduled for Mid-2. We'd rather show eight we can defend.

Q. What if the whole approach doesn't work?

> Each risky item has a fallback written before the result existed. If reproduction misses the band, we report both numbers. If the Recovery Controller fails its gate, resumption ships via controlled replay, which Tier 1 already delivers. If Repair Memory fails, it ships as an on/off ablation study.

Part 12 — Corrections already applied to the PoC

Five inconsistencies were found in the file and **all five are fixed**. This is the record — read it, because the reasoning behind each one is likely panel material.

#	What it said before	What it says now	Why it mattered
1	Step 9: *"culprit: **Refund Calculator**, step 3"* — while step 10 fixed the promo table	Step 9: *"**origin: Promo Table (stale v2)**, surfaced at Refund Calculator."* The Promo Table node now lights up too	**HIGH.** The calculator subtracted correctly from a stale input. Blaming where a value *appears* rather than where it *originated* is the exact mistake Granite exists to prevent
2	Step 11: *"→ **4/5 recovered**"*	Step 11: *"→ **recovery rate measured**. $19.75."*	No spike produced 4-out-of-5. Spike 5 was one deterministic replay; Spike 8 was 4/6. The protocol point survives without an unsupported number
3	Spike 4 = **"demonstrated"** on Evidence, **"inconclusive"** on Silent Failure	**"partial"** on both, detail line now says *"**mechanism** flags known-wrong…"*	Two tabs graded one spike differently. "Partial" is defensible in one word
4	*"Complex computing characteristics — 8 cleared"*, ninth silently absent	*"**8 of 9** cleared"* + an amber note naming the ninth, rating it weak, and giving its date	A missing ninth looks like concealment when discovered. A named, dated, self-rated weak ninth looks like self-assessment
5	*"outside its benchmark's own two"*	*"never run on a multi-agent framework absent from its own benchmark ecosystem"*	ErrorProbe used three benchmarks and three backbone families. "Two" was not defensible

Also updated: the Silent Failure mechanism card now cites `poc_spike4b_mechanism_output.json` instead of `risk-register R10`. That test has its own committed output file now — citing the file beats citing a register entry.

Verified after the fixes: page renders, 12 nodes and 13 edges intact, all six tabs load, animation runs 0→11, Spike 4 reads "partial", the ninth-characteristic note displays.

Still genuinely open — not fixable by editing

Item	Status
CCP characteristic 6 — diverse stakeholders	Real gap. Closing it needs an actual conversation with an engineer who runs multi-agent systems, recorded. Scheduled Mid-2
Step accuracy 13.2% vs 59.5%	Real gap. Closing it is first-semester work
Silent-failure occurrence rate	Unmeasured, and likely to stay so. Injected recall is the measurable version
Recovery Controller	No PoC. Gated at Mid-1
Verified Repair Memory	One test, failed. Gated at Mid-1

None of these should be hidden. Each is already stated on the PoC or in this guide, and each has a prepared answer in Part 11.

Part 13 — Coverage checklist

Every element in the file, and where it is explained.

Global

- [x] Header logo, title, project code — §2.1
- [x] "Stream A" chip — §2.1
- [x] "8 spikes on disk" chip — §2.1
- [x] "$0 committed" chip + the misreading risk — §2.1
- [x] Six-tab bar, switching behaviour, scroll-to-top — §2.2
- [x] Evidence bars animate once per load — §2.2

Tab 1 — Overview

- [x] Hero "Agents fail. Silently." — §3.1
- [x] "Run the pipeline" button — §3.2
- [x] "Open execution graph" button — §3.2
- [x] Six mini-pipeline rows, red row 2, green row 6 — §3.2
- [x] Side notes: 4 agents, no signal, 0% API, agent + step, N=5, verified — §3.2
- [x] 71.4% agent accuracy — §3.3
- [x] 13.2% step accuracy + why it is shown — §3.3
- [x] 4/6 repair success + shared cause — §3.3
- [x] 8/8 spikes + honest negative — §3.3
- [x] "What" card — diagnosis layer — §3.4
- [x] "Why" card — 14.2% baseline — §3.4
- [x] "Novel" card — the combination — §3.4

Tab 2 — Execution

- [x] "Simulated from Spike 3b + 4 + 5 + 8" banner — §4.1
- [x] "Simulated run" badge — §4.1
- [x] Target-system band — §4.2
- [x] Granite band — §4.2
- [x] Intake Agent — §4.3
- [x] Policy Checker — §4.3
- [x] Refund Calculator — §4.3
- [x] Response Writer — §4.3
- [x] Output ("ships either way") — §4.3
- [x] Promo Table (v2 stale) — §4.3

- [x] Trace Store — §4.3
- [x] Graph Builder — §4.3
- [x] Constraint Check — §4.3
- [x] Causal Tracer — §4.3
- [x] Repair Agent — §4.3
- [x] Verified Replay — §4.3
- [x] Zero-LLM-call pattern — §4.3
- [x] Owner tags (Noor / Saif) — §4.3
- [x] All 12 animation steps — §4.4
- [x] The arithmetic (29.75 / 10.00 / 19.75 / 20.00) — §4.4
- [x] Play / Pause — §4.5
- [x] Step — §4.5
- [x] Reset — §4.5
- [x] Scrub slider + counter — §4.5
- [x] Zoom in / out — §4.5
- [x] Drag-to-pan — §4.5
- [x] Packet dots + colour coding — §4.6
- [x] Node inspector + dimming — §4.7
- [x] Legend (agent / tool / fault / verified) — §4.8
- [x] Culprit-vs-origin correction — §4.9
- [x] Replay-step correction — §4.9

Tab 3 — Silent Failure

- [x] All five passing checks — §5.2
- [x] Why fluency proves nothing — §5.2
- [x] Constraint formula block — §5.3
- [x] "Feed a wrong value" → FLAG — §5.4
- [x] "Feed a correct value" → PASS — §5.4
- [x] Why the PASS path matters — §5.4
- [x] Mechanism-test card — §5.5
- [x] Natural-occurrence card — §5.5
- [x] The two-runs distinction — §5.5
- [x] Scope-of-claim card, abstention — §5.6

Tab 4 — Evidence

- [x] Five run bars, all values — §6.1
- [x] Why five runs — §6.1
- [x] Step-index bug caption — §6.1
- [x] Published-accuracy chart — §6.2

- [x] AgenTracer range footnote — §6.2
- [x] "40.5 points still wrong" — §6.2
- [x] All eight spike rows — §6.3
- [x] Self-correction finding (3/3b) — §6.3
- [x] Honest negative (7) — §6.3
- [x] Control run (6) — §6.3
- [x] Spike 4 status correction — §6.3

Tab 5 — Comparison

- [x] All six matrix rows — §7.1
- [x] Four capability columns — §7.1
- [x] ✓ / ✗ / ? meanings, "?" discipline — §7.1
- [x] Gold Granite row is conceptual — §7.1
- [x] Column click filter + all four notes — §7.2
- [x] Closest paper card — §7.3
- [x] Closest products card — §7.3
- [x] Closest prior FYP card — §7.3
- [x] ErrorProbe wording correction — §7.3

Tab 6 — Scope

- [x] All eight CCP characteristics — §8.1
- [x] The ninth characteristic, now on the page — §8.1
- [x] Shahoud's lane + if-removed — §8.2
- [x] Noor's lane + if-removed — §8.2
- [x] Saif's lane + if-removed — §8.2
- [x] All five gates, Gate 1 highlighted — §8.3
- [x] Tier 1 = 8 — §8.4
- [x] Tier 2 = 2, with the two weak items named — §8.4
- [x] Extensions = 8, not promised — §8.4
- [x] "Pre-registered" meaning — §8.4

Deliverables

- [x] End-to-end explanation — Part 9
- [x] Timed presentation script — Part 10
- [x] Panel questions with answers — Part 11
- [x] Corrections-applied record — Part 12

Every interactive control, node, number, card, tag and claim in `Granite_Interactive_PoC.html` is covered above.